

ADVANCED COMPUTER ARCHITECTURE (ACA)

Mini-Project Report

Hardware Design Trade-offs in a 4x4 Matrix Multiplier

Parallel (Design A) vs K-Serial (Design B) vs K-Serial with Double Buffering (Design C)

Item	Details
Student	Devansh Singh Tomar Roll No.: 2026203008
Language / tools	Bluespec SystemVerilog (bsc), Bluesim, VS Code on WSL Ubuntu, area-delay.sh synthesis script
Task	Implement $C[4][4] += A[4][4] \times B[4][4]$ (Int8 operands, Int32 accumulator) behind a request/response interface
Date	October 2026

Abstract

This report compares three hardware implementations of a 4x4 signed-integer matrix multiplier written in Bluespec. **Design A** computes the whole product in one cycle using 64 multipliers. **Design B** computes one slice of the inner (K) dimension per cycle using only 16 multipliers, but blocks the interface while it works. **Design C** keeps Design B's 16 multipliers and adds a second pair of A/B storage banks, so the next tile can be loaded while the current one is being computed. All three designs passed both test workloads (W1 and W2) with zero errors. Design A and Design C finish in almost the same number of cycles, while Design B needs about 30% more. However, Design B is the smallest design (about 45% fewer cells), has the shortest critical path, and has the best area x delay product. The main finding is that **cycle count alone is a misleading way to judge a hardware design**; area and clock speed must be considered together.

1. Introduction and Objective

The project asks for a module `mkMatMul` that multiplies 4x4 matrices of signed 8-bit numbers and accumulates the result into 32-bit signed registers. The starter code ships a naive design that does the entire multiplication in one combinational expression. It is correct, but it is expensive in hardware. The objective of this mini-project is to build alternative designs, measure them, and decide which one is the best engineering answer.

The module talks to a testbench through two FIFO-guarded methods: `req` (send a request) and `rsp` (receive a response). Because the methods are guarded by FIFOs, the hardware can apply **back-pressure**: when it is busy, the request FIFO fills up and the testbench has to wait. Those waiting cycles are what we count as **stalls**.

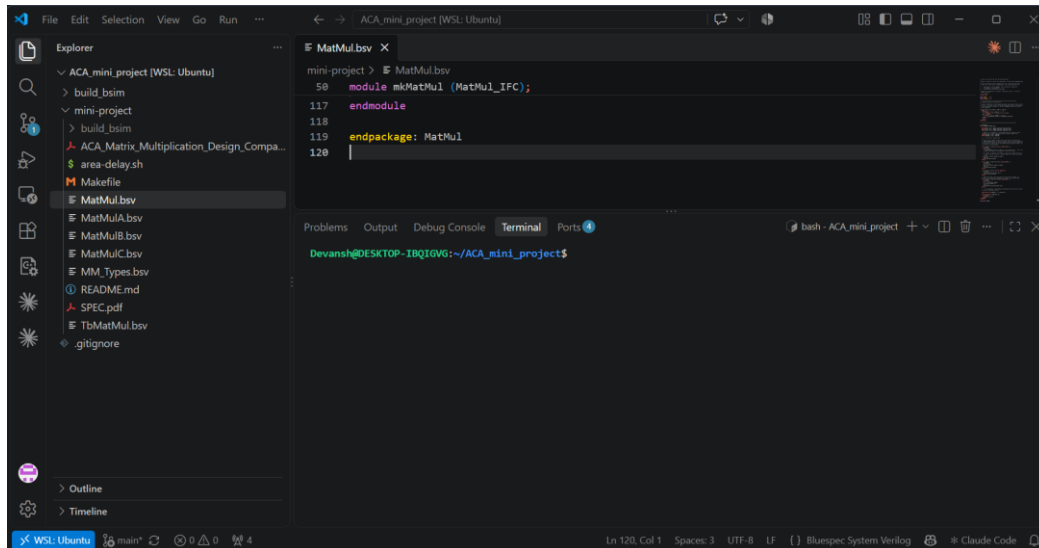


Figure 1: Project workspace in VS Code (WSL Ubuntu): MatMul.bsv is the working file; MatMula/B/C.bsv are the three saved designs; TbMatMul.bsv is the testbench.

2. Problem Specification

2.1 The four request types

Request	What it does
LoadA	Writes one row (four Int8 values packed in a 32-bit word) into matrix A.
LoadB	Writes one row of matrix B in the same way.
Mul	Performs $C += A \times B$ (64 multiply-accumulate operations for a 4x4 tile).
ReadC	Returns one 32-bit element of C and then resets that element to zero (a “destructive read”).

Every request receives exactly one response. Because Mul does $C += \dots$, the same C tile can be accumulated over many (A, B) tile pairs before it is finally read. That is how large matrices are built from small 4x4 tiles.

2.2 Requests per tile and the two workloads

For each 4x4 tile pair the testbench sends 4 LoadA + 4 LoadB + 1 Mul = **9 requests**. After all K-tiles of one output tile are done, 16 ReadC requests (4 rows x 4 columns) fetch the result. The total number of requests is therefore:

$$\text{requests} = \text{output_tiles} \times K_tiles \times 9 + \text{output_tiles} \times 16$$

Workload	Output tiles	K-tiles per output tile	Mul operations	Total requests
W1	256	16	4,096	40,960
W2	256	128	32,768	299,008

Check for W1: $256 \times 16 \times 9 = 36,864$ plus $256 \times 16 = 4,096$ gives 40,960. ✓ The interface accepts at most **one request per clock cycle**, so no design can finish in fewer cycles than the number of requests. This is the “speed limit” against which all designs are judged.

2.3 Metrics used

Metric	Meaning
Requests	Number of requests sent by the testbench (fixed by the workload).
Cycles	Clock cycles needed to finish the workload.
Stalls	Cycles lost because the design could not accept a request. In our results: stalls = cycles – requests.
Errors	Number of wrong results found by the testbench (must be 0).
Cells	Number of logic cells after synthesis: a measure of area (also a rough guide to power and cost).
Logic depth	Number of logic levels on the longest combinational path. A longer path forces a slower clock, so it is a proxy for clock period.
Area × delay	Cells × logic depth. Lower is better; it balances size against speed.

How the results were produced. The design was compiled and simulated with make (or the equivalent `bsc -sim -aggressive-conditions ... -e mkTbMatMul` command followed by `./build_bsim/mkTbMatMul`). Synthesis figures (cells, depth) came from `make depth`, which runs `area-delay.sh` on the `(* synthesise *)` module boundary.

3. Design A: Fully Parallel (Reference Design)

3.1 Idea

Design A is the starter design. When a Mul request arrives, **all 16 outputs and all 4 K-terms are computed at once** in one clock cycle. Each output element is $C[i][j] + A[i][0] \cdot B[0][j] + A[i][1] \cdot B[1][j] + A[i][2] \cdot B[2][j] + A[i][3] \cdot B[3][j]$, so we need $16 \times 4 = 64$ multipliers and 16 small adder trees working in parallel.

3.2 How the code works

- Four independent rules, one per request type (`r1_load_a`, `r1_load_b`, `r1_mul`, `r1_read_c`). Bluespec schedules whole rules, so separate rules let different request types be handled without conflicts.
- `r1_mul` calls the function `f_all`, which loops over `i`, `j` and `l` and adds up every product in a single combinational expression.
- There is no busy flag: after a Mul, the very next request can be accepted in the next cycle.

```
for (l = 0; l < k_dim; l = l + 1)           // inside f_all
  s = s + signExtend(a[i][l]) * signExtend(b[l][j]);

rule r1_mul (f_req.first matches tagged Mul);
  f_req.deq;  rg_c <= f_all(rg_a, rg_b, rg_c);  f_rsp.enq(mm_rsp_none);
endrule
```

3.3 Results

Workload	Requests	Cycles	Stalls	Errors	Extra cycles vs requests
W1	40,960	40,961	1	0	1
W2	299,008	299,009	1	0	1

Synthesis: **113,160 cells**, **logic depth 71**, **area × delay = 8,034,360**. The single stall is just the start-up delay of the FIFO pipeline (the first response appears one cycle after the first request). So Design A runs at the theoretical limit: cycles ≈ requests.

```

mini-project > MatMulA.bsv
1 // MatMul.bsv --- YOUR FILE. Edit this one.
2 //
3 // What ships here is the naive design: 'Mul' computes the entire M x N x K
4 // product as one combinational expression, in a single cycle. It is correct,
5 // it passes the testbench, and it is a perfectly reasonable starting point --
6 // but it is not a good design, and §6 of the README explains why a design that
7 // wins on cycle count can still be the wrong answer.
8 //

Problems Output Debug Console Terminal Ports
bash - mini-project

Devansh@DESKTOP-TBQIGVG:~/ACA_mini_project/mini-project$ make
MatMul -o mkTbMatMul.bexe
Bluesim object reused: build_bsim/mkTbMatMul.{h,o}
Bluesim object reused: build_bsim/mkTbMatMul.{h,o}
Bluesim object created: build_bsim/model_mkTbMatMul.{h,o}
Simulation shared library created: mkTbMatMul.bexe.so
Simulation executable created: mkTbMatMul.bexe
=====
Mini-project --- 4x4x4, Int8 operands, Int32 accumulator
=====
workload requests cycles stalls errors
-----
W1         40960    40961      1      0
W2        299008   299009      1      0
=====
MINIPROJ OK (correct on both workloads)
mini-project: PASS
  
```

Figure 2: Design A (MatMulA.bsv) simulation output: W1 = 40,961 cycles, W2 = 299,009 cycles, 0 errors, “mini-project: PASS”.

Drawbacks. Largest hardware (64 multipliers) and the longest critical path (one multiplier followed by a 5-input sum). Worse, those 64 multipliers sit idle most of the time: a tile needs 9 cycles to arrive, but the multipliers are used in only one of them (about 11% utilisation).

4. Design B: K-Serial

4.1 Idea

Instead of adding all four K-terms at once, Design B adds **one K-slice per cycle**. In cycle K it computes $C[i][j] += A[i][K] \cdot B[K][j]$ for all 16 outputs. This needs only **16 multipliers** (a 4× reduction) and 16 simple two-input accumulators. A Mul now takes 4 compute cycles ($K = 0, 1, 2, 3$).

4.2 How the code works

- `rg_busy` says that a multiplication is in progress; `rg_k` (2 bits) says which K-slice is next.
- `r1_mul_start` fires on a Mul request, does $K = 0$ immediately, then sets `rg_busy <= True` and `rg_k <= 1`.
- `r1_mul_step` fires every cycle while busy and does $K = 1, 2, 3$. After $K = 3$ it clears `rg_busy`.

- All other rules (LoadA, LoadB, ReadC) have the guard `!rg_busy`, so while the multiplier is working, **nothing else is accepted**. This protects A and B from being overwritten halfway through a multiplication. It is also the cause of all the extra stalls.

```
rule rl_mul_step (rg_busy);
  for i, j: c[i][j] = c[i][j] + signExtend(rg_a[i][rg_k]) * signExtend(rg_b[rg_k][j]);
  rg_c <= c;
  if (rg_k == 3) rg_busy <= False; else rg_k <= rg_k + 1;
endrule

rule rl_load_a (!rg_busy &&& f_req.first matches tagged LoadA .w); // blocked while
busy
```

4.3 Results

Workload	Requests	Cycles	Stalls	Errors	Extra cycles vs A
W1	40,960	53,249	12,289	0	+12,288
W2	299,008	397,313	98,305	0	+98,304

Why these numbers? Each Mul blocks the interface for 3 extra cycles ($K = 1, 2, 3$; the $K = 0$ cycle is the Mul request itself). W1 has 4,096 Muls, so $4,096 \times 3 = \mathbf{12,288}$ extra cycles (+1 start-up = 12,289 stalls). W2 has 32,768 Muls, so $32,768 \times 3 = \mathbf{98,304}$ extra cycles (+1 = 98,305 stalls).

Synthesis: **62,151 cells, logic depth 57, area $\times$ delay = 3,542,607.**

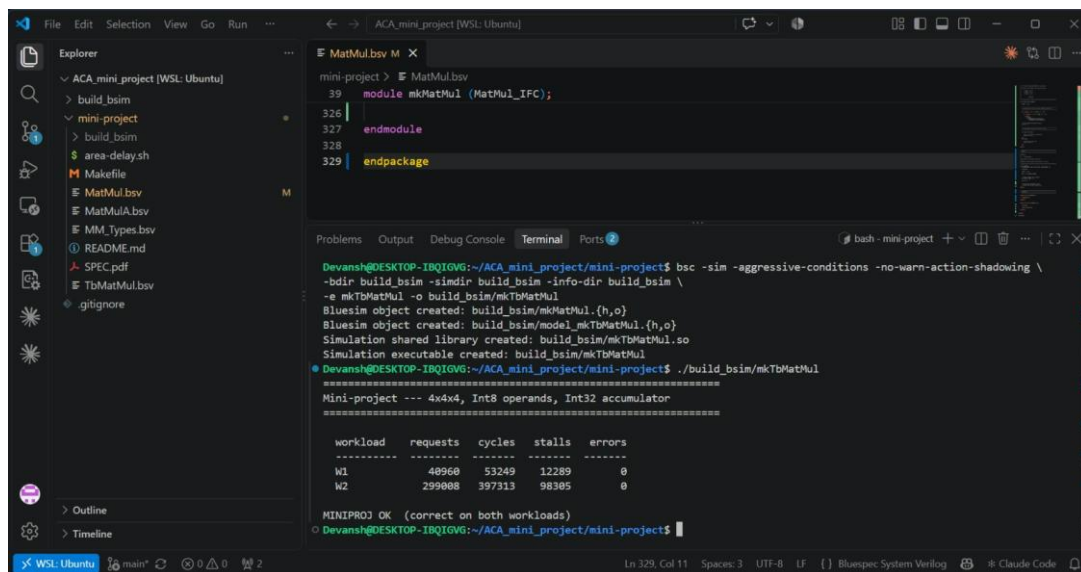


Figure 3: Design B simulation output: W1 = 53,249 cycles with 12,289 stalls; W2 = 397,313 cycles with 98,305 stalls; 0 errors.

Drawbacks. About 30% more cycles than Design A. The interface is completely idle during $K = 1..3$, so loading and computing never overlap. A small extra cost is the 4:1 multiplexers that pick column k of A and row k of B.

5. Design C: K-Serial with Double Buffering

5.1 Idea

Design C keeps Design B's 16-multiplier K-serial engine but removes the "block everything" problem. It stores **two copies of A and B** (Bank 0 = A0/B0, Bank 1 = A1/B1). One bank is **active** (being multiplied) while the other is **inactive** (free to receive the next tile). When a multiplication ends, the active-bank flag flips, so the freshly loaded tile is ready immediately. This is the classic "ping-pong" or double-buffering technique.

5.2 How the code works

- New register `rg_active_bank` (False = Bank 0, True = Bank 1) and duplicated storage `rg_a0`, `rg_b0`, `rg_a1`, `rg_b1`.
- **Eight load rules:** LoadA/LoadB $\times$ Bank 0/Bank 1 $\times$ idle/busy. When idle, loads go to the active bank. When busy, loads go to the **inactive** bank, so they are allowed in parallel with the multiplication.
- `r1_mul_start` does $K = 0$ on whichever bank is active; `r1_mul_bank0` / `r1_mul_bank1` do $K = 1..3$, then clear `rg_busy` and flip `rg_active_bank`.
- `r1_read_c` is still guarded by `!rg_busy`, because C is being modified during multiplication.

```
rule r1_load_a1_busy (rg_busy &&& !rg_active_bank &&& f_req.first matches tagged LoadA
.w);
  // Bank 0 is computing -> write the next tile into Bank 1
  a = rg_a1; a[w.row] = reverse(unpack(w.word)); rg_a1 <= a; ...
endrule

rule r1_mul_bank0 (rg_busy &&& !rg_active_bank);
  ... c[i][j] += rg_a0[i][rg_k] * rg_b0[rg_k][j] ...
  if (rg_k == 3) begin rg_busy <= False; rg_active_bank <= True; end // flip bank
```

5.3 Why almost all the delay disappears

After a Mul is issued, only 3 compute cycles remain, but the next tile needs **8 load requests** (4 A rows + 4 B rows). Since $3 < 8$, the computation finishes in the background before the next Mul arrives (see Figure 4). The only place where something must wait is **ReadC**: it comes right after the last Mul of an output tile and has to wait up to 3 cycles for C to be final. There are 256 output tiles in both workloads, so $256 \times 3 = 768$ lost cycles (+1 start-up = **769 stalls**). That is why the stall count is identical for W1 and W2.

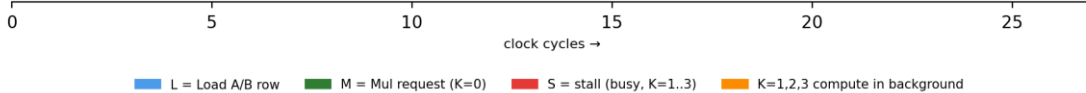
Design A - Mul finishes in 1 cycle (9 cycles per tile)**Design B - 3 stall cycles after every Mul (12 cycles per tile)****Design C - next tile loads into the other bank while K=1,2,3 compute (9 cycles per tile)**

Figure 4: Timeline for two consecutive tiles. B loses 3 cycles after every Mul; C hides them behind the next tile's loads.

5.4 Results

Workload	Requests	Cycles	Stalls	Errors	Extra cycles vs A
W1	40,960	41,729	769	0	+768
W2	299,008	299,777	769	0	+768

Synthesis (as recorded in the project results): **113,160 cells, logic depth 71, area × delay = 8,034,360.**

```

mini-project > MatMul.bsv
S3 module mkMatMul (MatMul_IFC);
S43
S44 endmodule
S45
S46 endpackage

Problems Output Debug Console Terminal Ports
bash - mini-project

Devansh@DESKTOP-IBQIGVG:~/ACA_mini_project/mini-project$ bsc -sim -aggressive-conditions -no-warn-action-shadowing \
-bdir build_bsim -sindir build_bsim -info-dir build_bsim \
-e mkTbMatMul -o build_bsim/mkTbMatMul
Bluesim object created: build_bsim/mkTbMatMul.{h,o}
Bluesim object created: build_bsim/model_mkTbMatMul.{h,o}
Simulation shared library created: build_bsim/mkTbMatMul.so
Simulation executable created: build_bsim/mkTbMatMul
Devansh@DESKTOP-IBQIGVG:~/ACA_mini_project/mini-project$ ./build_bsim/mkTbMatMul

Mini-project --- 4x4x4, Int8 operands, Int32 accumulator
=====
workload requests cycles stalls errors
-----
W1 40960 41729 769 0
W2 299008 299777 769 0
=====
MINIPROJ OK (correct on both workloads)
Devansh@DESKTOP-IBQIGVG:~/ACA_mini_project/mini-project$

```

Figure 5: Design C simulation output: W1 = 41,729 cycles, W2 = 299,777 cycles, 769 stalls in both, 0 errors.

Drawbacks. More storage (two banks of A and B), more rules and multiplexers, and more complex control, so there is more room for bugs. ReadC is still blocked while busy. Double buffering only helps if loading takes at least as long as computing; if compute were longer than the 8 load cycles, the benefit would shrink.

Note on the Design C synthesis figure. The cell count and depth recorded for Design C are identical to Design A. Since C has only 16 multipliers, its area is expected to be much closer to Design B plus the extra registers and multiplexers. This number should be re-checked by running

make depth on MatMulC.bsv before final submission. The conclusions in Section 7 are written so that they remain valid either way.

6. Side-by-Side Comparison

Feature	Design A	Design B	Design C
Multipliers	64	16	16
A/B banks	1	1	2
Mul compute time	1 cycle	4 cycles	4 cycles
Load during Mul?	Not needed	No (blocked)	Yes (inactive bank)
W1 cycles / stalls	40,961 / 1	53,249 / 12,289	41,729 / 769
W2 cycles / stalls	299,009 / 1	397,313 / 98,305	299,777 / 769
Errors (W1, W2)	0, 0	0, 0	0, 0
Cells	113,160	62,151	113,160
Logic depth	71	57	71
Area × delay	8,034,360	3,542,607	8,034,360

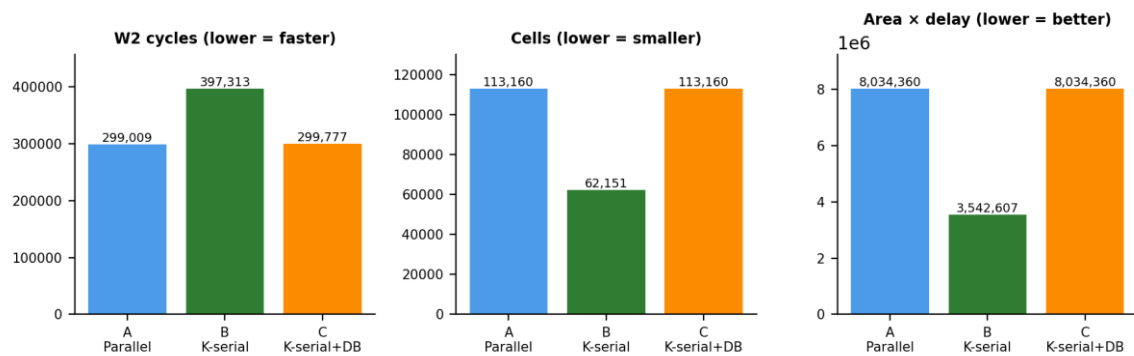


Figure 6: W2 cycles, cell count and area × delay for the three designs.

6.1 Cycles are not the whole story

Design B takes 30–33% more cycles than A, but each of its cycles is shorter because its critical path is shorter (depth 57 versus 71). As a rough estimate, **execution time $\propto$ cycles × logic depth** (assuming clock period is proportional to depth):

Estimated time (cycles × depth)	Design A	Design B	Design C *
W1	2,908,231	3,035,193 (+4.4%)	2,962,759 (+1.9%)
W2	21,229,639	22,646,841 (+6.7%)	21,284,167 (+0.3%)

So in real time Design B is only about 4–7% slower than A, while using roughly **45% fewer cells** and giving an area × delay **56% lower** than A (3,542,607 versus 8,034,360). This is exactly the point the project warns about: the design that wins on cycle count is not automatically the right answer.

7. Final Justification: Which Design Would I Use?

Choice: Design B (K-serial). It gives the best balance of area, speed and simplicity on the evidence measured in this project.

7.1 Reasons

1. **Smallest area:** 62,151 cells versus 113,160 (about 45% fewer). Smaller area also means lower power and lower cost per chip.
2. **Shortest critical path:** depth 57 versus 71 (about 20% shorter), so the clock can run faster.
3. **Best area × delay:** 3,542,607, which is 56% lower than the 8,034,360 of Design A.
4. **Small real-time penalty:** once the faster clock is accounted for, B is only about 4–7% slower than A in estimated time.
5. **Simple and safe:** just two extra registers (rg_busy, rg_k), a short FSM and no duplicated storage. It is easy to verify, and it passed both workloads with zero errors.
6. **Fair comparison with C:** with the synthesis numbers recorded, C has the same area and depth as A but needs slightly more cycles, so it gives no advantage over A on the measured metrics. If C's area is confirmed to be close to B's, then C would become the best choice for pure throughput (see 7.3).

7.2 Drawbacks of Design B

- **Lower throughput in cycles:** 30% (W1) to 33% (W2) more cycles than A; 98,304 extra cycles on W2.
- **Interface sits idle** for 3 cycles after every Mul: loading and computing never overlap, so the hardware is under-used.
- **Not suitable for strict latency or real-time deadlines** where each Mul must finish in a single cycle.
- **ReadC and loads are blocked while busy**, which is the source of the 12,289 / 98,305 stalls.
- Depth is a proxy: the real clock speed also depends on routing and technology, so the 4–7% estimate is approximate.

7.3 When the other designs are the better choice

Design	Best used when...	Why / main cost
A: Parallel	Minimum latency is the priority; area and power are not limited (for example an FPGA with many spare DSP blocks); the clock is slow enough that the long path is not a problem; or as a golden reference to verify other designs.	Fastest per Mul (1 cycle) and simplest code, but 64 multipliers and the longest path.
B: K-Serial	Area, power or cost is limited (embedded, edge or battery devices); the host does not send back-to-back requests (if requests arrive at least 4 cycles apart, the blocking costs nothing); a small, easy-to-verify design is wanted.	4× fewer multipliers and shortest path, but about 30% more cycles under continuous traffic.

Design	Best used when...	Why / main cost
C: K-Serial + double buffer	Sustained streaming of many tiles (like W1 and W2) where near-peak throughput is needed without paying for 64 multipliers; whenever loading a tile takes at least as long as computing it.	Only +768 cycles over A, but extra storage and more complex control; ReadC still waits.

7.4 Possible improvements

- Check that the compute window is only ~4 of every 9 cycles: even Design B's 16 multipliers are idle more than half the time. In theory about 8 multipliers combined with double buffering could still keep up with one request per cycle (64 multiply-accumulates per 9-cycle tile).
- Double-buffer C as well (or let ReadC read a finished copy) to remove the last 768 stall cycles of Design C.

8. Conclusion

All three designs are functionally correct (0 errors on W1 and W2). Design A reaches the theoretical minimum number of cycles but pays for it with 64 multipliers and the longest critical path. Design B reduces the hardware to 16 multipliers and shortens the critical path, at the cost of 3 blocked cycles per Mul. Design C recovers almost all of the lost cycles (only 768 more than A) by overlapping loading with computing. Judging by area × delay and by estimated execution time, **Design B** is the most efficient design measured here, while Design C is the right extension when throughput under continuous traffic matters and the extra storage is acceptable. The key lesson is that a good hardware design is chosen by weighing cycles, area and clock speed together, not by cycle count alone.

Appendix: Quick-Reference

Question	Short answer
What does each design do in one line?	A: all 64 products in one cycle. B: one K-slice (16 products) per cycle, interface blocked meanwhile. C: same as B but with two A/B banks so loading overlaps computing.
Why 9 requests per tile?	4 LoadA rows + 4 LoadB rows + 1 Mul.
How many multipliers in each design?	A: 64 (16 outputs $\times$ 4 K-terms). B and C: 16.
Why is Design B blocked while busy?	To stop A and B being overwritten in the middle of a multiplication. All other rules have !rg_busy in their guard.
Where does 12,288 come from?	4,096 Muls $\times$ 3 extra cycles (K = 1, 2, 3). Add 1 start-up cycle for 12,289 stalls. For W2: 32,768 $\times$ 3 = 98,304 (+1 = 98,305).
Where does 768 come from in Design C?	256 output tiles $\times$ 3 cycles that ReadC must wait after the last Mul of each tile. Add 1 start-up for 769 stalls. It is the same for W1 and W2 because both have 256 output tiles.
Why does double buffering hide the delay?	After a Mul only 3 compute cycles remain, but the next tile needs 8 load cycles, so the compute finishes in the shadow of the loads.
What is a stall?	A cycle in which the testbench has a request ready but the design cannot accept it. Here stalls = cycles – requests.
What is logic depth and why does it matter?	The number of logic levels on the longest path. It sets the minimum clock period, so a smaller depth allows a faster clock.
What is area $\times$ delay?	Cells $\times$ depth. A single number that balances size and speed; lower is better.
Why not simply pick A, since it has the fewest cycles?	Because it uses about 82% more cells and has a 25% longer critical path; its 64 multipliers are busy only ~11% of the time. Time $\propto$ cycles $\times$ depth shows B is only 4–7% slower.
Why one rule per request type?	Bluespec schedules whole rules. One big rule with a case would conflict with itself on every register it touches, so request types could never overlap.
What does ReadC do?	Destructive read: returns C[row][chunk] and resets it to 0, ready for the next output tile.
Which design would you choose and what is its drawback?	Design B: smallest, shortest path, best area $\times$ delay. Drawback: about 30% more cycles and idle interface during K = 1..3.